

OpenMP

High Performance Computing
Nuno Lopes

Motivation

- ▶ Library to develop parallel applications using shared memory.
- ▶ High level abstraction that allows adaptation of sequential applications in a simple way.
- ▶ Pthreads library is more complex in terms of API.

Pragma Mechanism in C

- ▶ OMP commands are based on pragmas.
- ▶ Pragmas allow you to add functionality to a C program, without invalidating its generic compilation.
- ▶ `# pragma omp <...>`
- ▶ Compilers that support these directives make use of them.
- ▶ Compilers that do not support these directives ignore them and compile the program as usual.
- ▶ Correctly coded OpenMP program always compiles on any platform, even without support.

Compiler Extensions for OpenMP

- ▶ Auxiliary Library “omp.h”

```
#include <omp.h>
```

- ▶ Pre-compiler directive to validate support:

```
# ifdef _OPENMP
```

```
...
```

```
#else
```

```
...
```

```
#endif
```

Compilação e execução de programas OpenMP

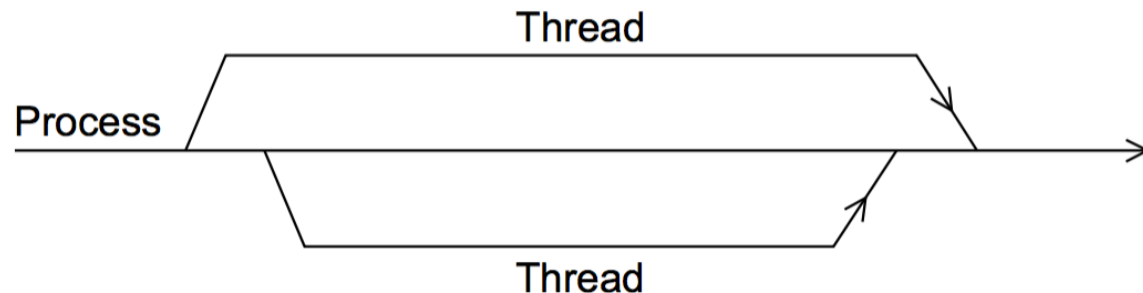
- ▶ Compiling option:
 - ▶ -fopenmp
 - ▶ gcc -fopenmp hello_omp.c -o hello_omp
- ▶ Linker option:
 - ▶ -lomp
 - ▶ Optional if including -f

Parallel Directive

```
# pragma omp parallel
```

```
<code>
```

- ▶ Creates multiple threads of execution, threads, in the same process, which each execute the code immediately following the pragma.
- ▶ When the code is finished, all threads wait for the completion of the remaining threads.



Parallel Directive

► Code:

```
# pragma omp parallel
printf("Esta é a thread %d, num threads %d\n",
omp_get_thread_num(), omp_get_num_threads());
printf("Fim\n");
```

► Result:

```
Esta é a thread 2, num threads 3
Esta é a thread 1, num threads 3
Esta é a thread 0, num threads 3
Fim
```

numthreads Option

```
# pragma omp parallel num_threads(thrcnt)  
<code>
```

- ▶ *numthreads()* defines *thrcnt* number of threads to parallelise the application.
- ▶ If not specified, the number of threads created is set by the system during execution, e.g. number of cores.

Auxiliary Functions in Library

- ▶ `int omp_get_thread_num(void)`

Returns the identifier of the calling thread.

- ▶ `int omp_get_num_threads(void)`

Returns the number of existing threads.

- ▶ `omp_set_num_threads(int)`

Sets number of threads to use.

- ▶ `OMP_NUM_THREADS=4`

Environment variable, which allows to define number of threads.

- ▶ Option `-lomp` allows to compile code to access these functions, even without the OpenMP parallelism.

Parallel For Directive

```
# pragma omp parallel for
for( int i=0; i<N; i++)
    printf("Contador i %d na thread %d, #threads %d\n",
```

- ▶ Allows parts of the cycle to be parallelised, in an automated way, by several threads.
- ▶ Cycle counter variable is private for all threads.
- ▶ Restrictions:
 - ▶ Number of iterations must be able to be calculated at the beginning of the cycle, and keep constant during execution.
 - ▶ Cannot alter the execution flow (ex: break)

Parallel For Directive

► Result:

```
Contador i 7 na thread 2, num threads 4  
Contador i 1 na thread 0, num threads 4  
Contador i 4 na thread 1, num threads 4  
Contador i 9 na thread 3, num threads 4  
Contador i 8 na thread 2, num threads 4  
Contador i 2 na thread 0, num threads 4  
Contador i 5 na thread 1, num threads 4  
Contador i 10 na thread 3, num threads 4  
Contador i 3 na thread 0, num threads 4  
Contador i 6 na thread 1, num threads 4  
Fim
```